Caliper: Universal Sewing Machine Setup & Calibration Agent

Individual Project Proposal | Arya Devkota

1. Problem Statement & Motivation

What Real-World Business Problem Are You Addressing?

Mechanical garment construction and textile fabrication rely on precise physical relationships between fabric weave, elastane stretch percentages, thread tensile strength, and machine hardware calibration. Operating a mechanical sewing machine, such as the Brother SM3701, Singer Heavy Duty 4423, or Janome HD1000, requires configuring multiple interdependent controls: upper thread tension dials, stitch length and width selectors, needle gauge, and presser foot attachments. Because every fabric weight, measured in Grams per Square Meter (GSM), and stretch elasticity responds differently under needle penetration, improper mechanical calibration leads to severe stitch defects, including skipped stitches, thread bird-nesting, seam puckering, and torn fibers. In commercial apparel production, boutique garment alteration, and custom tailoring, trial-and-error machine setup wastes high-grade textile materials, consumes billable labor hours, and frequently damages delicate deadstock or synthetic stretch fabrics.

Who Are the Target Users?

The primary target users include independent garment makers, custom tailors, costume designers, apparel design students, and home sewists who operate mechanical or semi-digital sewing machines. These users regularly transition between diverse fabric mediums, ranging from 4-way spandex knits to heavy denim and delicate silk satin, and require immediate, error-free machine calibration tailored to their specific machine chassis without manually deciphering dense manufacturer reference manuals.

Why Is an Agentic AI Solution Appropriate Here?

Traditional automated workflows, such as fixed decision trees, hardcoded lookup scripts, or linear automated pipelines, depend on static input-output mappings. They fail in this domain because mechanical garment construction involves non-linear interactions across unindexed sewing machine models, variable fabric densities, elastane stretch percentages, and real-time physical stitch failure modes. An automated script cannot handle unexpected inputs or troubleshoot physical defects dynamically.

In contrast, an agentic AI solution brings non-linear reasoning, dynamic context ingestion, and closed-loop adaptability. Instead of breaking when encountering an unindexed machine model or complex fabric blend, an agentic system dynamically queries external hardware specifications, reasons across physical constraints, and autonomously invokes specialized tools to generate calibrated configurations.

Furthermore, unlike linear automated workflows that execute once top-to-bottom without feedback, an agentic multi-agent loop provides real-time diagnostic evaluation. It interprets natural language user descriptions of real-time stitch defect symptoms (such as underside looping, puckering, or needle deflection), isolates physical root causes, and autonomously adjusts machine parameters in a closed feedback loop until optimal stitch calibration is achieved. Finally, an agentic system acts as an active safety guardian, intercepting hazardous fabric-needle mismatches before execution rather than blindly running invalid parameters.

2. Use Cases & Scenarios

Scenario 1: Cross-Machine Calibration for Delicate Stretch Fabrics

A garment maker needs to sew a side seam on a lightweight 4-way polyester-spandex jersey knit (200 Grams per Square Meter (GSM), 50% stretch) using a Brother SM3701. The user launches the Caliper application, opens the machine catalog selection modal, and chooses the Brother SM3701. Upon setting the machine context, the user enters the fabric specifications and target seam type into the chatbot interface. The Machine Ingestion Agent parses the Brother SM3701 hardware profile from the repository data store, identifying its 0 to 9 tension dial scale and snap-on presser foot system. The Fabric Mechanics Agent calculates required thread elasticity, and the Calibration Agent generates and executes a Python calibration script to compute exact dial settings. The Visualization Agent then renders the precise configuration on an interactive machine dial panel: Organ Ballpoint 75/11 needle, upper tension set to 3.5, Stretch Zig-Zag stitch (Dial C, length 2.5mm, width 1.5mm), and Zigzag Foot J. When the user later switches to a Singer Heavy Duty 4423, Caliper dynamically re-calibrates the output to Singer's specific dial positions (Tension 4.0, Pattern S1-B).

Scenario 2: Real-Time Stitch Defect Diagnosis & Dynamic Python Correction

While sewing a woven cotton linen hem, a tailor encounters severe thread looping on the underside of the fabric (bird-nesting). The user types the defect description directly into the Caliper chatbot. The Safety & Diagnostic Critic evaluates the mechanical failure mode, identifying a primary imbalance between upper thread tension and bobbin resistance. The Critic triggers the Calibration Agent to re-run the tension ratio equation with an elevated friction coefficient in the Python Read-Eval-Print Loop (REPL) runtime. The system verifies the root cause, instructs the user to check that the presser foot was raised during upper threading, and the Visualization Agent updates the visual dial interface to show a revised upper tension setting from 3.0 to 4.5, resolving the looping without trial-and-error testing on scrap fabric.

Scenario 3: Fabric-Needle Physical Constraint Verification

A sewist attempts to configure a heavy 14oz denim flat-felled seam using a fine 70/10 Universal needle. Upon receiving the user input, the Safety & Diagnostic Critic intercepts the request before machine settings are rendered. The Critic evaluates needle penetration force limits against fabric density and identifies an immediate risk of needle deflection and shank breakage. Caliper halts execution, issues a safety warning in the chat stream, and automatically substitutes the setup with a

reinforced 100/16 Jeans needle, an upper tension of 6.0, and a heavy-duty straight stitch length of 3.5mm.

3. Initial System Design

Multi-Agent Architecture & Specialized Roles

Caliper employs a 5-node agentic workflow implemented in LangGraph. The architecture decouples hardware ingestion, physical mechanics, algorithmic code execution, visual rendering coordination, and safety evaluation into specialized autonomous agents. To maintain structural execution control and token efficiency across zero-cost endpoints (Groq REST Application Programming Interface (API) and Google AI Studio Software Development Kit (SDK)), the workflow incorporates XML Grounding within LangGraph. Rather than incurring high token overhead through JSON function calling, agents anchor their operational routing intent inside explicit XML tags. A lightweight Python regular expression parser extracts these tags to trigger zero-token conditional edge transitions in LangGraph. Furthermore, each agent incorporates an automated internal audit function that executes post-task verification to ensure operational accuracy:

Agent Role	Primary Responsibilities	Internal Audit Tool
Machine Ingestion Agent	Parses user machine model, extracts hardware dial bounds (0-9 vs 1-5), stitch pattern maps, and presser foot specs.	validate_machine_schema(): Audits hardware range definitions and JSON schema validity.
Fabric Mechanics Agent	Analyzes fabric GSM weight, weave structure, and elastane stretch percentage to determine structural seam requirements.	verify_fabric_physics(): Audits fabric density, stretch boundaries, and thread strain limits.
Calibration Agent	Generates and executes Python calibration scripts in a REPL runtime to calculate exact non-linear dial coordinates.	audit_calibration_code(): Conducts Abstract Syntax Tree (AST) analysis and execution checks on code.
Visualization Agent	Coordinates visual rendering of machine dials, knob angles, presser feet, and control layouts for the front-end UI.	verify_visual_coordinates(): Validates SVG and Canvas dial angles, knob bounds, and UI state mapping.
Safety & Diagnostic Critic	Enforces physical safety constraints (needle size vs GSM) and diagnoses stitch defect failure modes.	audit_safety_constraints(): Audits physical hazard rules and verifies defect recommendations.

Expected Tools, APIs, and Data Sources

Caliper's front-end application architecture is designed for maximum deployment flexibility. The application sits in a public GitHub repository and can be cloned and run locally as a desktop web app. Alternatively, it can be deployed using a Serverless Frontend approach on GitHub Pages or

Vercel, running client-side JavaScript that calls external inference APIs directly (including an API key settings box where users input their own keys). It can also be deployed via a Full-Cloud Hybrid Architecture, hosting the user interface on Vercel or GitHub Pages while routing multi-agent loops to a free Python backend hosted on Render or Railway. On initial launch or login, users are prompted via a searchable catalog modal to select their sewing machine model (such as the Brother SM3701, Singer Heavy Duty 4423, or Janome HD1000) or skip to set it later, tailoring the chatbot session specifically to that hardware chassis.

Resource / Integration	Technical Functionality & Scope	Implementation Method
GitHub Repository & Internal API	Serves as the central code repository and data store for machine hardware specifications, JSON schemas, needle compatibility matrices, and factory baselines.	GitHub REST API and Local JSON Data Store
Sandboxed Python REPL Runtime	Executes dynamically generated Python calibration scripts to compute exact numerical dial positions without LLM math hallucination.	Local Sandboxed Python 3.12 REPL Tool
Free LLM API Router	Zero-cost inference via Groq Llama 3.3 70B and Google AI Studio Gemini 2.5 Flash endpoints, utilizing dual-homing failover during rate-limit events.	Groq REST API and Google AI Studio SDK
Web Search & Scraping Tools	Dynamically fetches unindexed sewing machine manuals, manufacturer baseline charts, or community troubleshooting guides when encountering new models.	DuckDuckGo API and BeautifulSoup Web Scraper
Front-End Application & Dial Visualizer	Flexible multi-tab application architecture designed for local execution or cloud hosting (GitHub Pages, Vercel, or Render/Railway), featuring an interactive chatbot and visual dial canvas.	HTML5, CSS3, JavaScript, and Web Storage APIs

4. Feasibility Statement

I would like to emphasize that I believe my project, Caliper, is fully feasible for me to complete by the end of this semester due to my professional background as a full-time Data Engineer and my extensive experience architecting complex agentic systems, data pipelines, and API integrations. The technical scope of Caliper is strictly bounded around deterministic physical rules and machine specifications, eliminating the need for costly model fine-tuning or heavy backend infrastructure. By embedding XML Grounding directly into LangGraph nodes, the system achieves extreme token efficiency and keeps its core container footprint under 40MB, allowing it to run reliably on 100% free hosting tiers like Render or Hugging Face without running out of memory. Combined with zero-cost API dual-homing and sandboxed Python code execution, Caliper's core deliverables align cleanly with the course milestone schedule.

Potential Challenges & Mitigation Strategies

Risk Category & Challenge	Severity / Impact	Mitigation Strategy
Numerical Hallucination on Dial Coordinates	High (Physical Machine Misconfiguration)	Restricts LLMs to code generation, delegating all numerical calculations to executed Python scripts in the REPL runtime.
Free-Tier LLM API Rate Limits (HTTP 429)	Medium (Service Interruption)	Implements an automated dual-homing API router that seamlessly fails over from Groq Llama 3.3 to Gemini 2.5 Flash upon encountering rate limits.
Python REPL Code Execution Errors	Medium (Workflow Stoppage)	Wraps generated calibration code in AST syntax checks and try-except blocks, falling back to deterministic manufacturer baseline rules if execution fails.
Hardware Schema & Unindexed Models	Low (Incomplete Machine Context)	Utilizes a standardized JSON hardware schema format on GitHub, backed by web scraping tools to ingest unindexed machine manuals on demand.
Physical Needle & Fabric Damage	High (Material Loss or Broken Needle)	The Safety & Diagnostic Critic intercepts invalid setups before visual dial rendering, enforcing physical safety boundaries.

Project Execution Timeline

The project execution timeline strictly adheres to the course schedule and milestone deadlines:

Phase / Date	Milestone Deliverable	Technical Focus & Objectives
Week 6 (Oct 1)	Project Proposal (10%)	Problem definition, user scenarios, 5-agent LangGraph workflow design, and feasibility statement.
Week 9 (Oct 22)	Milestone I: Front-End (5%)	Interactive web UI with machine selection catalog modal, chatbot interface, and dynamic visual dial panel.
Week 12 (Nov 12)	Milestone II: Multi-Agent (5%)	Complete LangGraph 5-agent workflow with Calibration Agent Python REPL execution, internal audit functions, and dual-homing API router.
Week 13 (Nov 19)	Milestone III: Evaluation (5%)	Behavioral evaluation matrix testing stitch defect resolution accuracy, code execution validity, and safety guardrail enforcement.
Week 14 (Dec 2-3)	Final Presentation & Report (30%)	10-minute video demonstration, pilot evaluation results, and comprehensive technical final report.